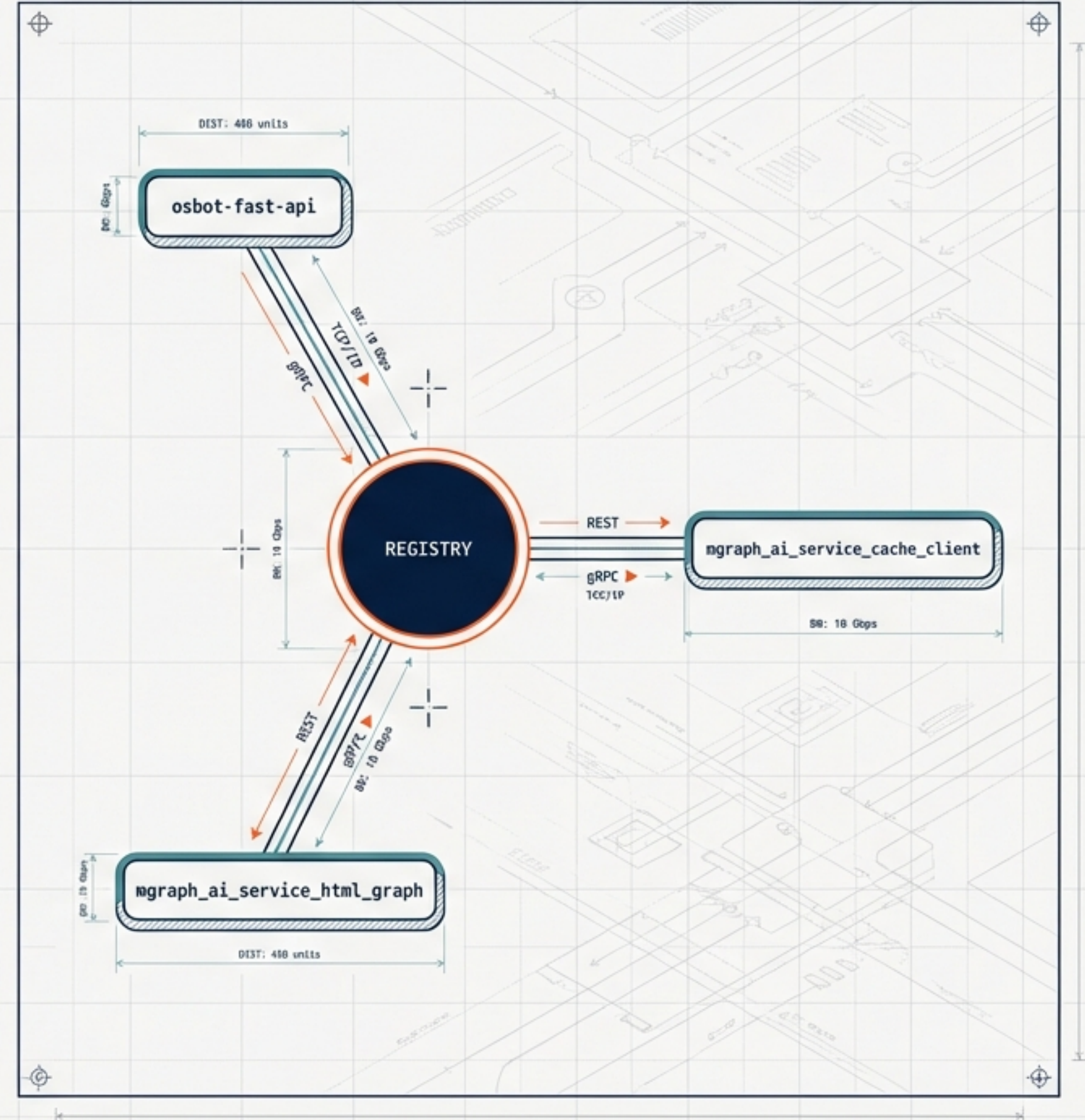


Unified Service Client Architecture

TECHNICAL DEBRIEF V0.34.0 | JANUARY 2025

Internal Engineering Brief



Executive Summary

Refactored service clients to utilise a Centralised Registry Pattern with Stateless Facades.

~15

Files Deleted

Across two major service packages

80%

Less Boilerplate

Reduction in test setup code

0

Duplication

Zero duplicate configs or handlers

The Big Win: Introduced **multi-service in-memory composition**—wiring complex service dependencies without network overhead.

The New Standard for Integration Testing

Two lines. Two services. Fully wired. In-memory.

```
from mgraph_ai_service_cache.helpers.Run_Cache_Service import  
    register_cache_service__in_memory  
from mgraph_ai_service_html_graph.helpers.Run_Html_Graph_Service  
import register_html_graph_service___in_memory  
  
register_cache_service__in_memory()  
register_html_graph_service__in_memory()
```

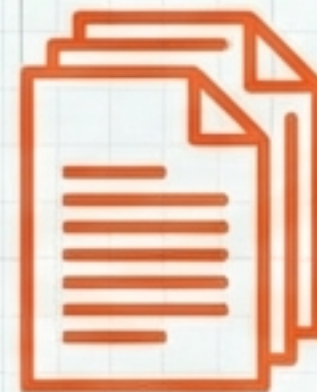
Previously, this required individual configuration, bespoke wrapper classes, and complex manual orchestration.

Why We Refactored: The Cost of Stateful Clients



Stateful Clients

Each client held its own config, making them difficult to swap or mock.



Boilerplate Overload

10+ lines of setup code required per test class.



Duplication

Config schemas and request handlers copy-pasted across every service.

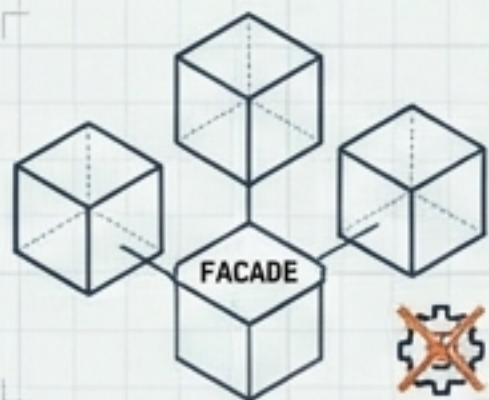


The Wiring Nightmare

Connecting dependencies (e.g. Graph + Cache) required manual orchestration in every test.

Design Philosophy: Centralised & Stateless

Stateless Clients



Stateless Clients

Pure facades with no config.
Lookups happen at request time.

JetBrains Mono

Centralised Registry

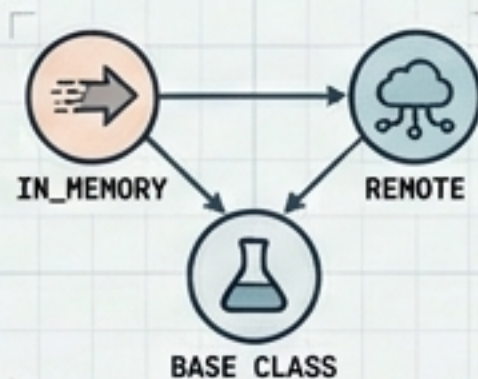


Centralised Registry

The single source of truth for all active configurations.

JetBrains Mono

Generic Transport

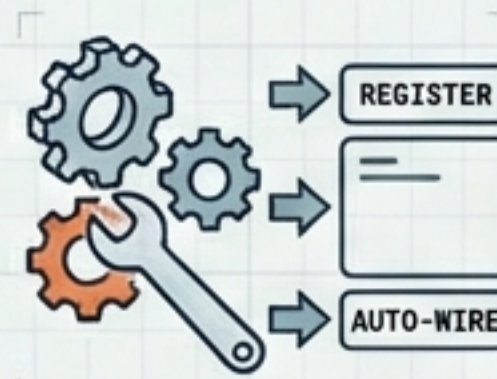


Generic Transport

One base class handling both IN_MEMORY and REMOTE execution.

JetBrains Mono

Registration Helpers

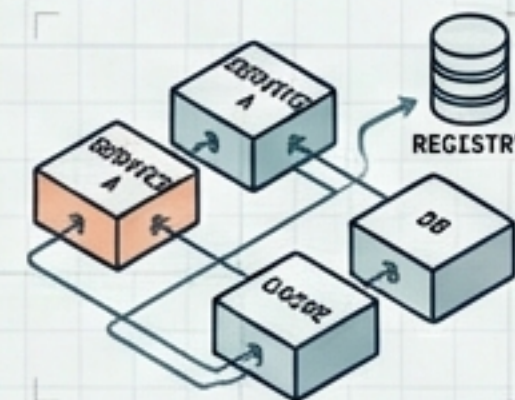


Registration Helpers

Standardised functions to handle the wiring automatically.

JetBrains Mono

Composition

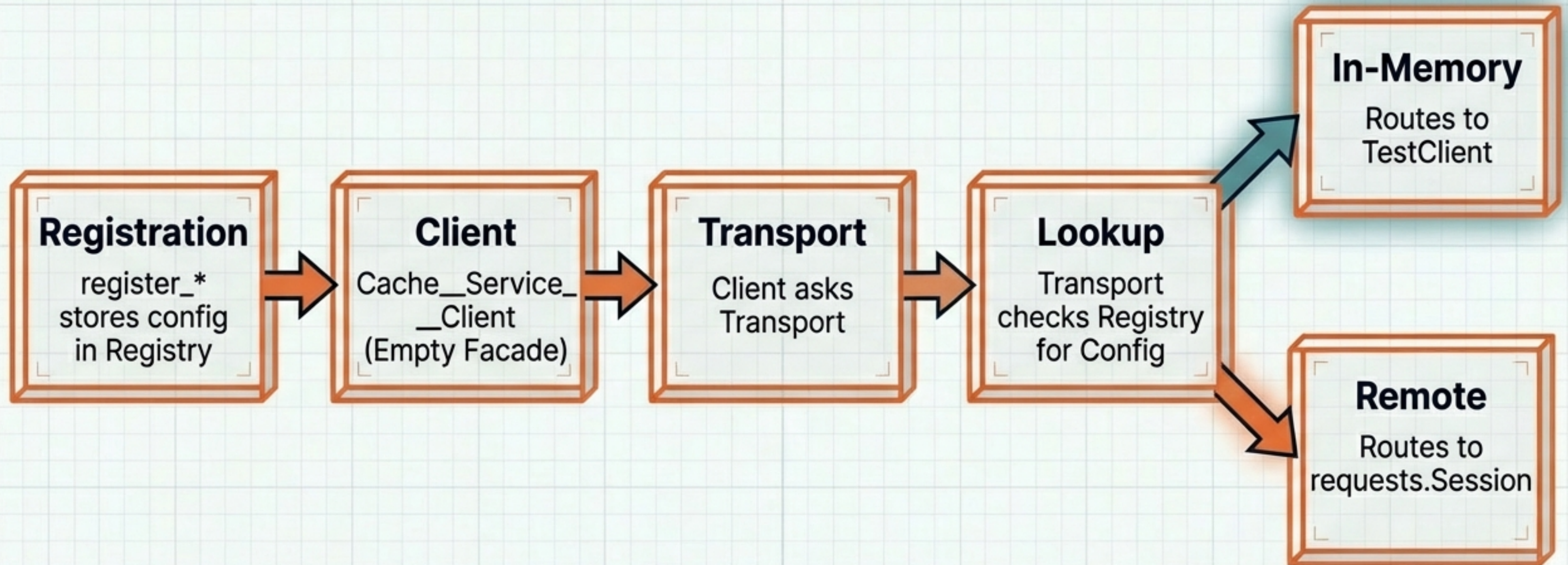


Composition

Services discover dependencies via the shared registry.

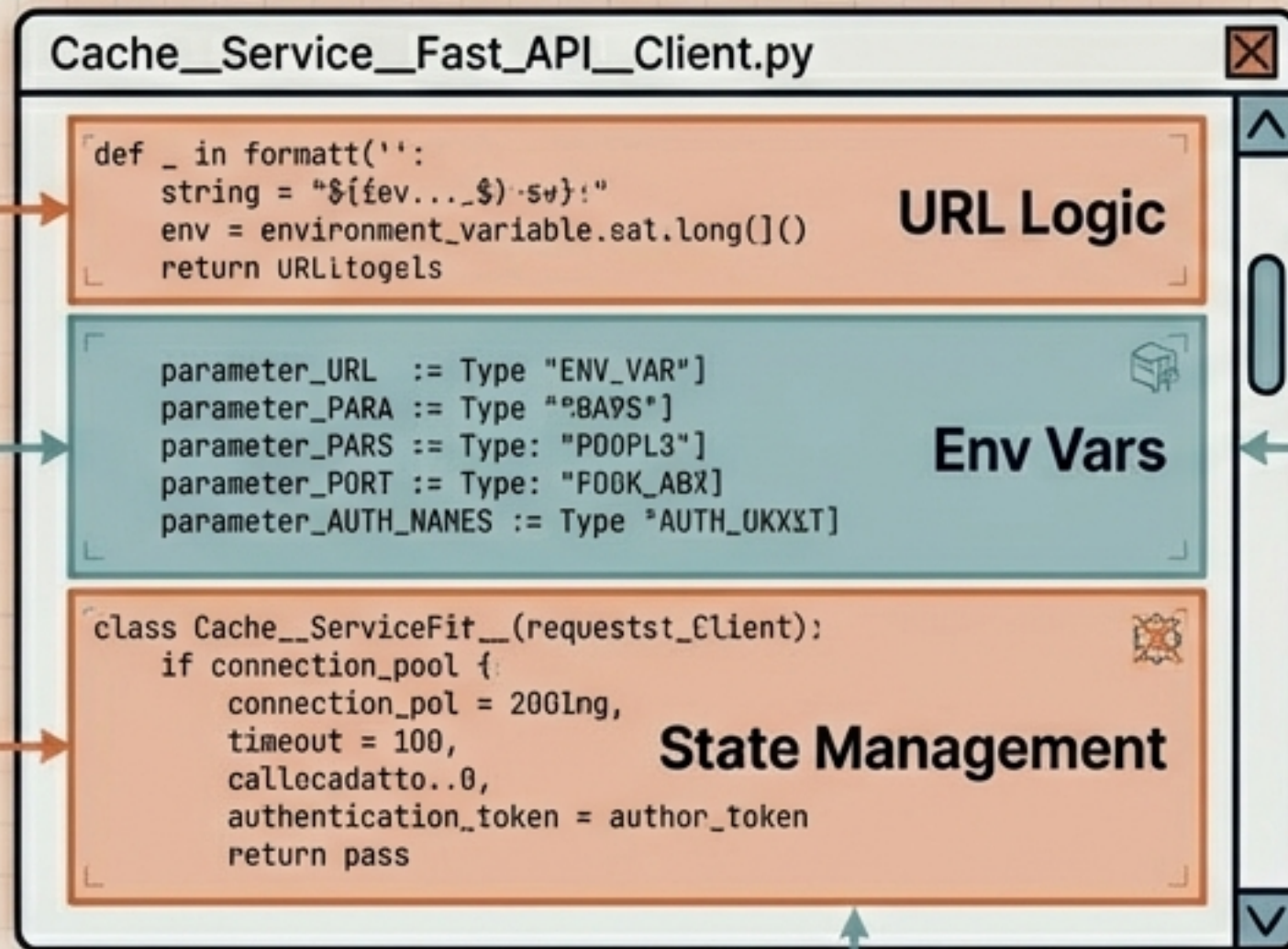
JetBrains Mono

Request Lifecycle



From 200 Lines to Zero State

OLD (Stateful)



Heavy config logic mixed with transport.

NEW (Stateless)

The diagram shows a code editor window titled 'Cache_Service_Client.py'. It contains a single class definition:

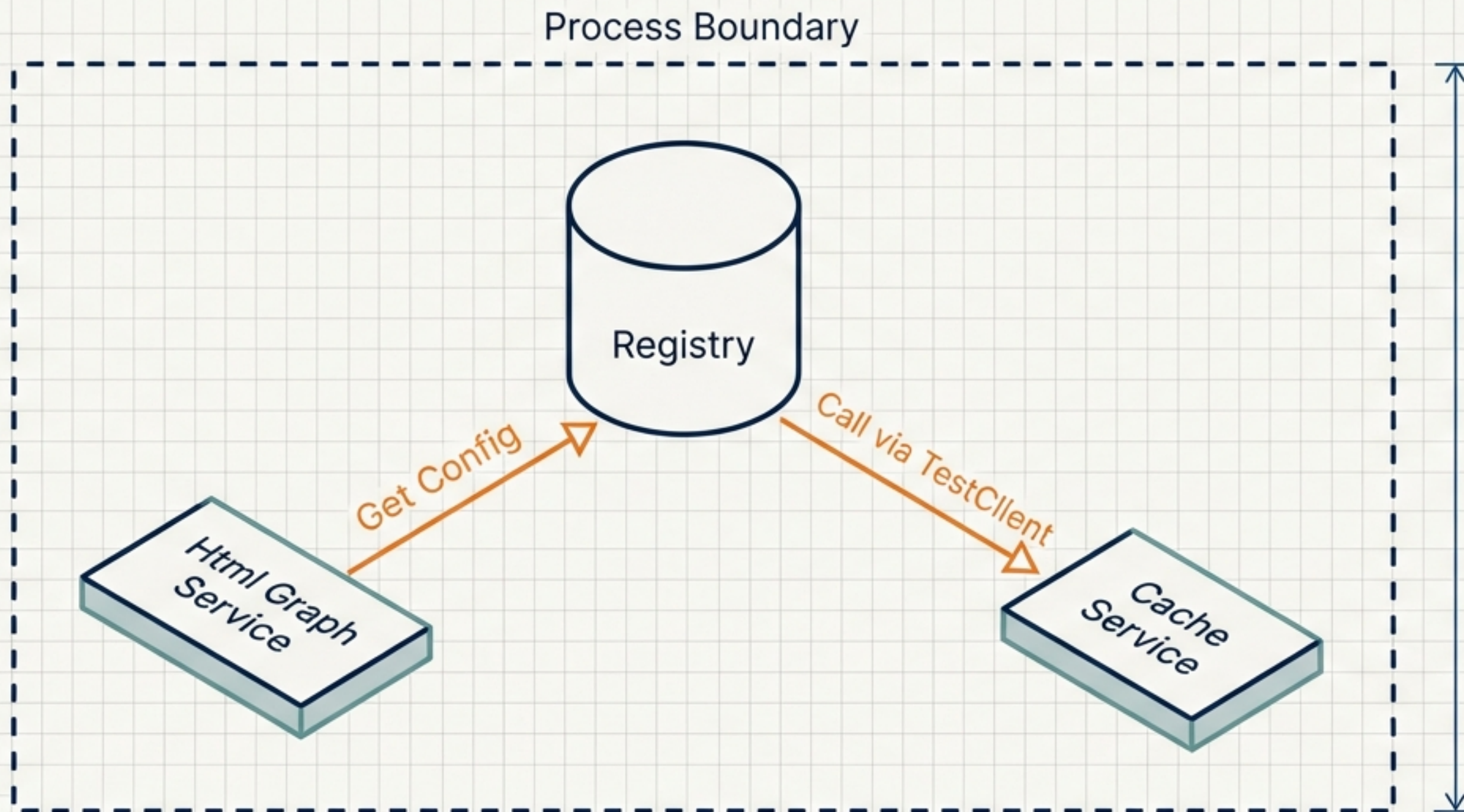
```
class Cache_Service_Client(Fast_API_Client_Requests):  
    pass
```

An arrow points from the text 'Empty shell inheriting base capabilities.' to the class definition.

Empty shell inheriting base capabilities.

Clients no longer know *how* to connect; they only know *what* to ask for.

Seamless Service Interdependency



Scenario: Html Graph depends on Cache.

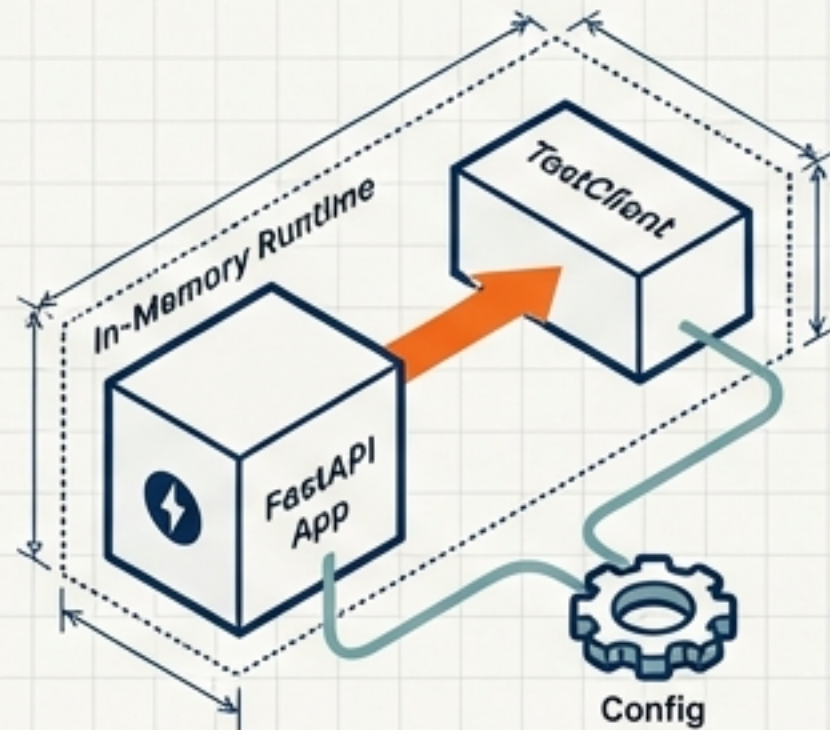
Mechanism: Both register to the same registry. Communication is direct, in-memory, without network ports.

Standardised Entry Points

Decoupling test environments from environment variables.

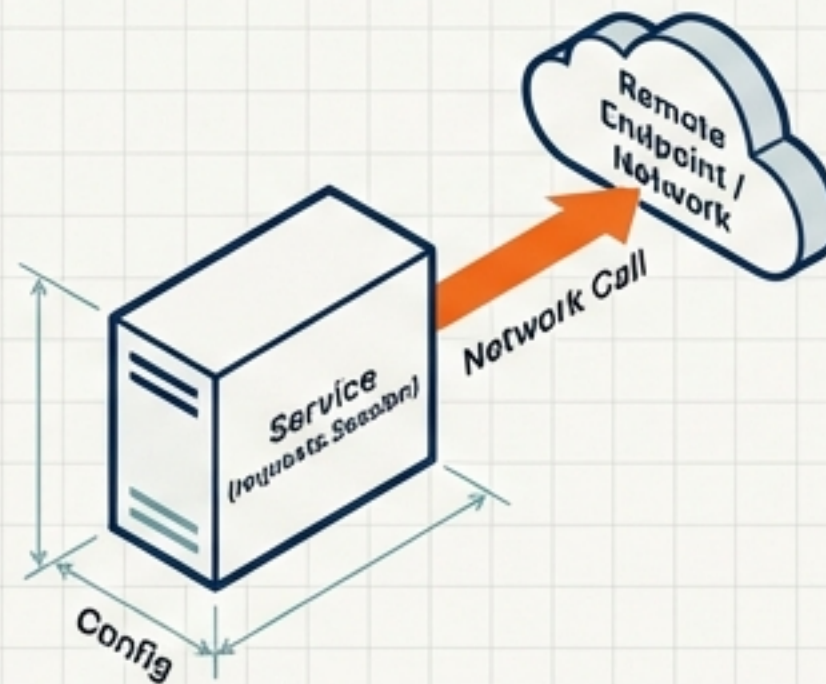
`register_service__in_memory()`

Creates the FastAPI app, wires the TestClient, and registers config.



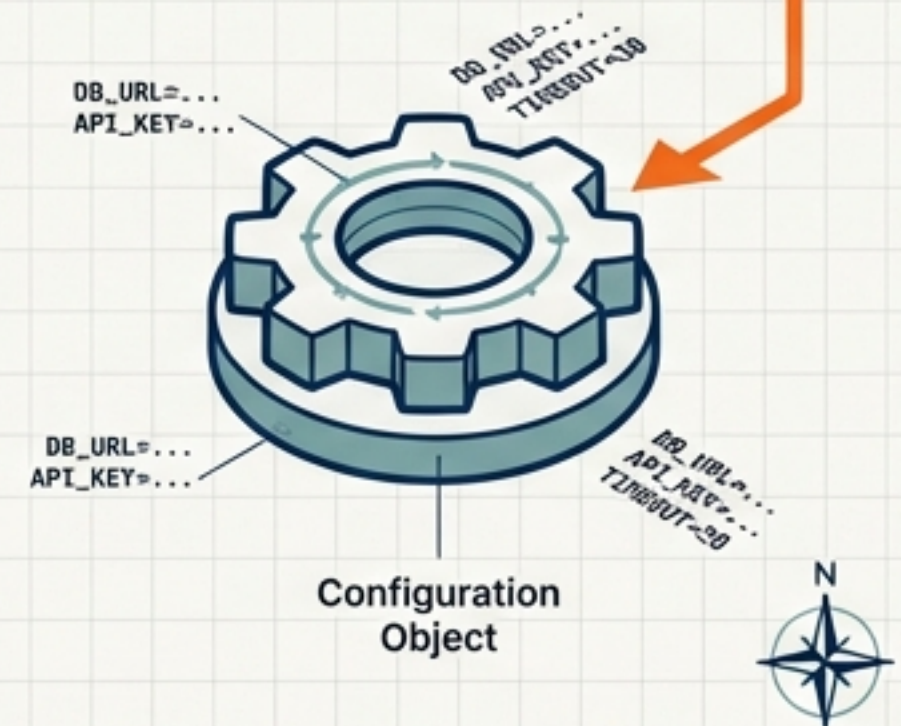
`register_service__remote()`

Configures the service for live network calls via requests.Session.



`service_config()`

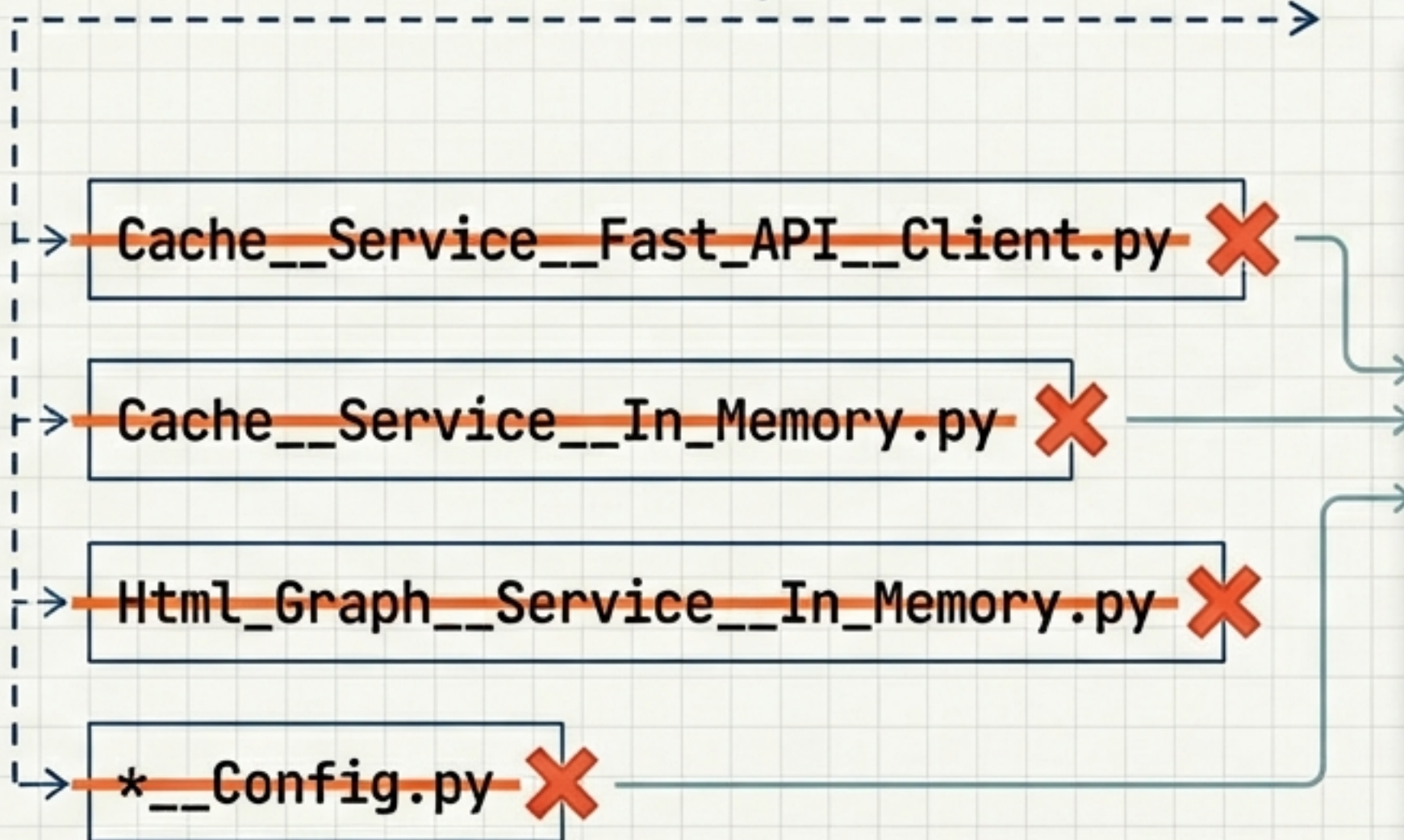
Provides direct access to the configuration object.



Addition by Subtraction

Removing legacy components and simplifying the test architecture.

The Graveyard

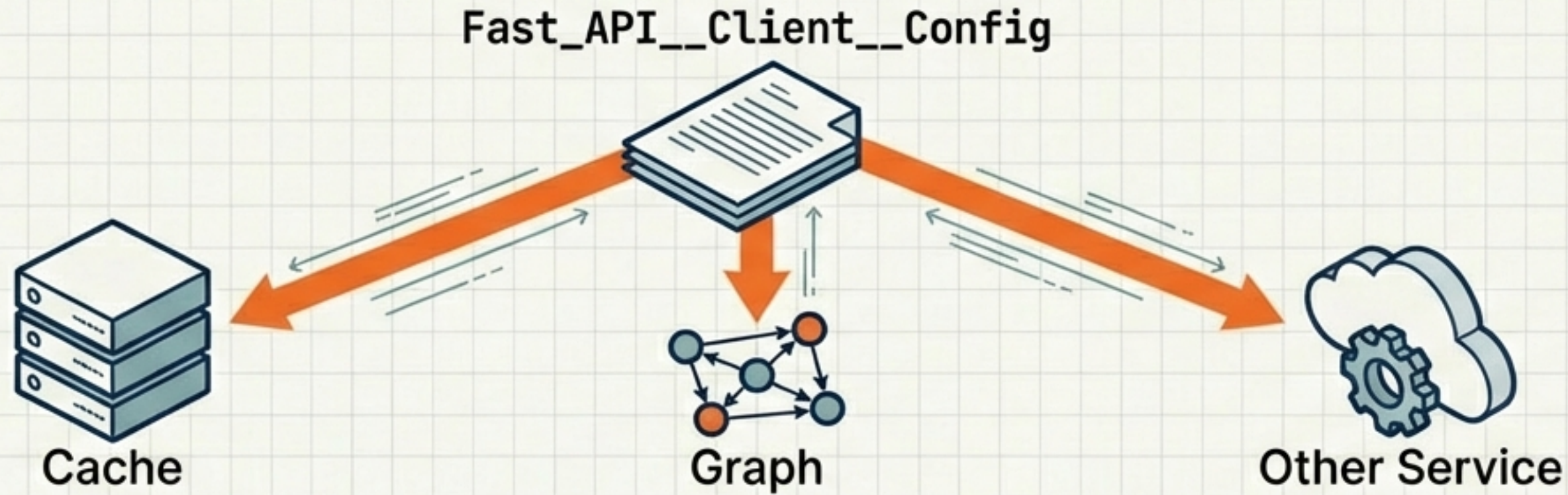


The Data

Component	Impact
Cache Client Code	↓ 60% Reduction
In_Memory Wrappers	X 100% Elimination
Test Setup	↓ 80% Reduction

A Single Source of Truth

Centralizing configuration schemas for consistency across services.



How do we handle service-specific settings (e.g. Namespaces)?

Moved to method parameters.

- No hidden state
- Explicit overrides per call
- Defaults provided for convenience

Qualitative Impact

	Before	After
Config Location	Scattered in each client	-----> Centralised in Registry
Transport Logic	Duplicated everywhere	-----> Single Base Implementation
Test Isolation	Manual Save/Restore	-----> Automatic via Registry
Env Coupling	Direct Env Var reads	-----> Abstracted by Helpers

Migration Guide

For Maintainers

For Testers

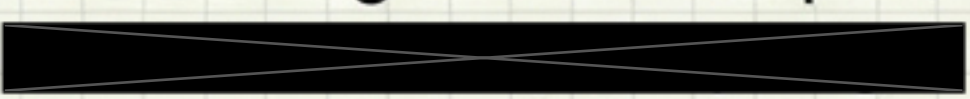
1. Create Stateless Client (inherit base)



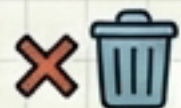
2. Create Transport (One-liner)



3. Create Registration Helpers



4. **DELETE** old files
(*__In_Memory.py, *__Config.py)



Find & Replace



`self.fast_api_client` → `self.service_client`

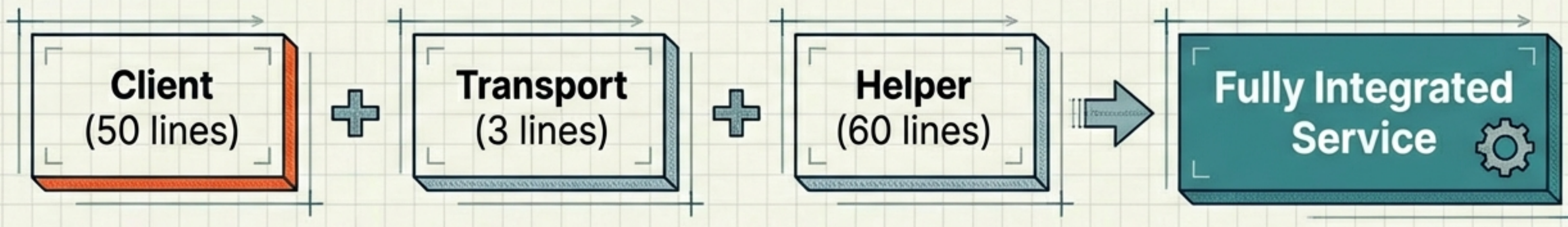
`Cache__Service__Fast_API__Client`
---→ `Cache__Service__Client`

`setup()` ---→
---→ `register *__in_memory()`

What This Enables Next

Immediate Win

Adding a new service (e.g., LLM Service) now takes minutes.



Future Potential



Summary

By centralising configuration, we transformed multi-service integration from a complex chore into a declarative 2-line process.



“We didn’t lose functionality by deleting 500+ lines—we gained it.”